STM32-BASED SMART SECURITY AND ACCESS CONTROL SYSTEM

PROJECT REPORT
submitted by

ANGEL ELSA MATHEW(SBC24CS059)
ASWATHY A(SBC24CS077)
ASWATHY R(SBC24CS079)
MERLIN JOSE(SBC24CS146)
SANJANA SATHEESAN(SBC24CS178)

submitted As part of project based learning course PBCST504

SEMESTER 5

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SREE BUDDHA COLLEGE OF ENGINEERING(AUTONOMOUS)

ALAPPUZHA - 690529

OCTOBER 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SREE BUDDHA COLLEGE OF ENGINEERING

(AUTONOMOUS)

PATTOOR P. O., ALAPPUZHA

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"STM32-Based Smart Security and Access Control System**" submitted by Aswathy A(SBC24CS077),Aswathy R(SBC24CS079),Angel Elsa Mathew(SBC24CS059),Merlin Jose(SBC24CS146),Sanjana Satheesan(SBC24CS178) in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of the work carried out under my guidance and supervision at college.

Ms. Aryalekshmi A
Course CO-Ordinator
(Assistant Professor)
Dept. of CSE

Dr. S Suresh Babu
Head of Department
Dept. of CSE

CONTENTS

ABSTRACT ..... **I**

LIST OF FIGURES ..... **II**

CHAPTER 1: INTRODUCTION

1.1 Project Overview 1

1.2 Objectives 2

1.3 Scope of the Project 3

1.4 Applications 4

CHAPTER 2: HARDWARE AND SOFTWARE REQUIREMENTS

2.1 STM32 Microcontroller Overview 5

2.2 STM32 Development Board and Specifications 6

2.3 Hardware Components and Sensors Used 7

2.3.1 STM32F103C8T6 Development Board 7

2.3.2 4×4 Keypad 7

2.3.3 LED Indicators 8

2.3.4 Power Supply 8

2.4 Software Requirements 9

2.4.1 Operating System 9

2.4.2 STM32CubeIDE 9

2.4.3 Embedded C Programming Language 10

2.4.4 STM32 Firmware and Configuration Tools 10

2.4.5 Device Programming and Debugging Tools 10

2.5 STM32CubeIDE and STM32 HAL Library 11

2.5.1 STM32CubeIDE 11

2.5.2 STM32 HAL Library 11

Advantages of Using STM32 HAL 12

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture 13

3.2 Block Diagram 13

3.3 Circuit Diagram 14

3.4 Hardware Component Description 15

3.4.1 STM32F103C8T6 Microcontroller 15

3.4.2 4×4 Matrix Keypad 15

3.4.3 Red LED 15

3.4.4 Green LED 15

3.4.5 Servo Motor 15

3.4.6 Power Supply 15

3.5 Pin Configuration and Peripheral Mapping 16

3.6 Algorithm and Flowchart 16

3.6.1 Algorithm 16

3.6.2 Flowchart 17

3.7 Working Principle 18

CHAPTER 4: IMPLEMENTATION

4.1 Development Environment Setup 19

4.2 STM32CubeMX Configuration 19

4.3 GPIO and Peripheral Configuration 19

4.4 Embedded C Program Development 20

4.5 Hardware Interfacing and Integration 20

4.6 Program Execution and Debugging 21

4.7 Implementation Challenges and Solutions 21

CHAPTER 5: TESTING AND RESULTS

5.1 Testing Methodology 23

5.2 Test Cases and Observations 23

5.3 Hardware Implementation Photographs 23

5.4 Output Screenshots 24

5.5 Result Analysis 24

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion 25

6.2 Learning Outcomes 25

6.3 Limitations 25

6.4 Future Enhancements 26

APPENDIX A: COMPLETE EMBEDDED C SOURCE CODE 27–37

APPENDIX B: ADDITIONAL CIRCUIT DIAGRAMS AND CONFIGURATION DETAILS

B.1 Circuit Connection Details 38

B.2 STM32CubeMX Configuration 38

B.3 Password Configuration 39

B.4 Additional Diagrams 39–40

ABSTRACT

Security and access control have become essential requirements in modern residential and commercial environments. Conventional door-locking systems often rely on traditional keys, which can be lost, duplicated, or misused. To overcome these limitations, this project presents the design and development of a Budget-Friendly Password-Based Smart Door Security System using the STM32F103C8T6 microcontroller. The proposed system provides a simple, cost-effective, and reliable approach to controlling door access through password authentication, motion detection, and audio-visual alerts. The system is developed using the STM32F103C8T6 microcontroller, a 4×4 matrix keypad, a Passive Infrared (PIR) motion sensor, a buzzer, and red and green Light Emitting Diodes (LEDs). The firmware is developed using Embedded C in the STM32CubeIDE environment, with peripheral configuration and initialisation performed using STM32CubeMX. The keypad serves as the primary user interface for entering the password, while the microcontroller verifies the entered password against the predefined authorised password. When the correct password is entered, the system grants access and indicates successful authentication using a green LED. When an incorrect password is entered, access remains restricted, and the system activates a red LED and buzzer to indicate an unauthorised access attempt. UART-based serial communication can also be incorporated for debugging and monitoring system events during development and testing. The proposed design focuses on affordability, simplicity, and ease of implementation, making it suitable for a low-cost security prototype. The system can also be extended with a servo motor and a suitable driver circuit to demonstrate automatic door opening and closing after successful authentication. Further improvements may include mobile notifications, wireless monitoring, password modification, access-attempt logging, and integration with Internet of Things (IoT) platforms. The project demonstrates the practical application of embedded systems, microcontroller programming, digital input/ output interfacing, sensor integration, and basic access-control mechanisms. By combining password-based authentication with motion detection and alert indications, the proposed system provides an affordable foundation for developing smarter and more accessible door security solutions.

LIST OF FIGURES

Figure 1.1:Overview of the Proposed Smart Door Security System……..…..2

Figure 2.1:Technical Specifications Table……...…….6

Figure 2.2:STM32F103C8T6 Development Board (Blue Pill)…….……...…7

Figure 2.3:Hardware Components Used in the Smart Door Security System……..……8

Figure 2.4:Hardware Summary Table……..9

Figure 2.5:Software Summary Table……...…….……10

Figure 3.1:Block Diagram of the Smart Door Security and Access Control System……...……14

Figure 3.2:Circuit Diagram of the STM32-Based Smart Door Lock System…….…….……...…..…14

Figure 3.3:Pin Configuration and Peripheral Mapping Table……...…….…….….16

Figure 3.4:Flowchart of the Password-Based Door Lock System…….…….……...……...…17

Figure 4.1:STM32 Pin Configuration and Component Functions…….…..…19

Figure 4.2:Pinout Configuration of STM32F103C8T6……...……20

Figure 4.3:Firmware Programming Using STM32CubeProgrammer…….……...……..…21

Figure 4.4:Password-Controlled Door-Lock Prototype…….……...… ..……22

Figure 5.1:System Test Cases and Expected Results…….……..…23

Figure 5.2:STM32CubeIDE Build Output……..……...……..…24

Figure B.1:STM32 Pin Configuration and Component Functions…….38

Figure B.2:Complete Circuit Connection Diagram……...…….….39

Figure B.3:STM32CubeMX Pin Configuration Screenshot…….….39

Figure B.4:TIM3 Timer Parameter Configuration……..…40

Figure B.5:Title Clock Configuration in STM32CubeMX…….……..…..40

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

Security is an important concern in residential buildings, offices, laboratories, and other areas where access needs to be controlled. Traditional locking systems generally depend on physical keys and provide limited electronic monitoring. Lost or duplicated keys may create security concerns, while conventional locks typically do not provide electronic alerts when an unsuccessful access attempt occurs. The development of embedded systems and microcontroller-based technologies has made it possible to design affordable electronic security solutions that provide additional access-control features.

The Smart Door Security System is a microcontroller-based security prototype designed to provide password-based access control, motion detection, and alert generation. The system uses the STM32F103C8T6 microcontroller as its main processing unit. It integrates a 4×4 keypad for password entry, a Passive Infrared (PIR) sensor for detecting movement near the entrance, LEDs for status indication, and a buzzer for audible alerts. The keypad allows a user to enter a predefined password. The microcontroller processes the entered password and compares it with the stored password. If the password is correct, the system indicates successful authentication using an LED. If the password is incorrect, the system can activate a buzzer and a warning LED to indicate an unsuccessful access attempt. This provides a basic electronic access-control mechanism without relying exclusively on a physical key. The PIR sensor detects changes in infrared radiation associated with movement within its detection area. When motion is detected, it sends a signal to the microcontroller for processing. This feature adds motion-detection capability to the system. However, the PIR sensor detects movement and does not identify individuals or independently determine whether a person is authorised. The STM32F103C8T6 processes the inputs and controls the output devices according to the programmed logic. The system can be programmed using embedded C in STM32CubeIDE, providing practical experience in microcontroller programming, sensor interfacing, digital input and output operations, and embedded system development.

The primary implementation focuses on password verification, motion detection, and visual and audible alerts. Automatic door opening can be incorporated as an additional feature using a suitable actuator, such as a servo motor, if it is included in the final hardware implementation. Overall, the project demonstrates how affordable electronic components and a microcontroller can be integrated to develop a simple security prototype.

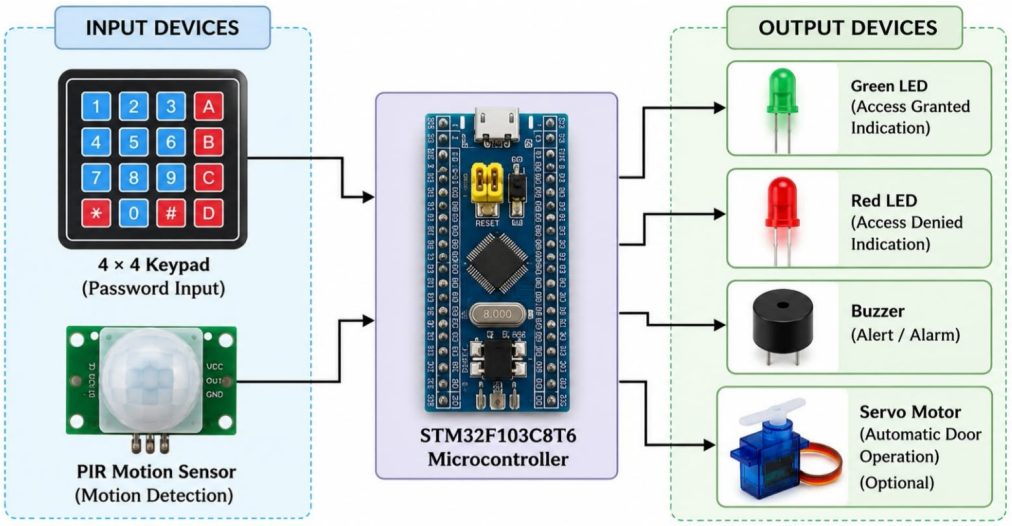


Figure 1.1: Overview of the Proposed Smart Door Security System

1.2 OBJECTIVES

The main objective of this project is to design and develop a low-cost, microcontroller-based smart door security system that provides basic electronic access control and motion detection. The specific objectives are:

1. To develop a smart door security prototype using the STM32F103C8T6 microcontroller.
2. To implement password-based authentication using a 4×4 keypad.
3. To verify the entered password through a programmed comparison mechanism.
4. To detect human movement near the entrance using a PIR sensor.
5. To provide visual indications using LEDs for different system states.
6. To generate audible alerts using a buzzer when an incorrect password is entered or a defined alert condition occurs.
7. To integrate the hardware components and software logic into a functional embedded system.

8. To test the system and evaluate its operation under different input conditions.
9. To provide a foundation for future enhancements, such as automatic door control and advanced authentication methods.

1.3 SCOPE OF THE PROJECT

The scope of the Smart Door Security System is to design and develop a low-cost, microcontroller-based security prototype that combines password-based access control, motion detection, and alert generation. The project demonstrates how embedded systems can be used to improve basic security through the integration of electronic components, sensors, and programmed control mechanisms. The STM32F103C8T6 microcontroller acts as the central processing unit of the system. A 4×4 keypad is used to enter a predefined password, which is compared with the stored password to determine whether authentication is successful. A Passive Infrared (PIR) sensor detects movement near the entrance and sends signals to the microcontroller for processing.

The system uses LEDs to indicate different operating conditions, such as successful authentication, incorrect password entry, and motion detection. A buzzer generates audible alerts when an incorrect password is entered or a predefined security condition occurs. These visual and audible indications help the user understand the system's status and identify potentially suspicious activity. The project includes hardware interfacing, embedded C programming using STM32CubeIDE, configuration of input and output pins, password verification, sensor signal processing, and control of output devices. Functional testing is carried out to evaluate the system's response under different conditions, including correct password entry, incorrect password entry, and motion detection.The basic implementation focuses on password authentication, motion sensing, and alert generation. Additional features, such as automatic door control using a servo motor, restrictions on repeated incorrect attempts, and security event logging, can be incorporated in future versions. Advanced features such as fingerprint authentication, facial recognition, internet connectivity, and remote monitoring are outside the scope of the current implementation.

Overall, the project provides practical experience in microcontroller programming, sensor interfacing, digital input and output control, and embedded system development. Its modular design allows additional components and features to be integrated in the future, making it a useful foundation for developing more advanced electronic access-control systems.

1.4 APPLICATIONS

The Smart Door Security System can be adapted for use in several environments where basic access control and motion detection are useful. Its affordability and simple design make it suitable for educational demonstrations and small-scale security prototypes.The potential applications include:

1. Residential Security: Can be used as a prototype for password-based access control at household entrances.
2. Office Rooms: Can demonstrate electronic access control for rooms with restricted entry.
3. Laboratories: Can be adapted for controlling access to laboratories or equipment rooms.
4. Educational Institutions: Can be used to demonstrate embedded systems, microcontroller programming, and sensor interfacing.
5. Storage Rooms: Can serve as a starting point for access-control systems in small storage areas.
6. Small Workshops: Can be adapted for basic access monitoring in workshops and similar environments.
7. Embedded Systems Projects: Provides a practical platform for learning keypad interfacing, motion sensing, password verification, and output-device control.
8. Future Smart Security Systems: Can be extended with additional sensors, improved authentication, event logging, or remote monitoring.

The proposed system is primarily an educational prototype. Before being used as a real security solution, it would require additional measures such as secure password storage, protection against repeated password attempts, reliable power management, and appropriate physical door-locking hardware

CHAPTER 2

HARDWARE AND SOFTWARE REQUIREMENTS

The Smart Door Security System consists of hardware and software components that work together to provide password-based access control, motion detection, and alert generation. The hardware includes the STM32F103C8T6 microcontroller development board, a 4×4 keypad, a PIR motion sensor, LEDs, a buzzer, and a suitable power supply. The software environment is used to configure the microcontroller, develop the embedded program, compile the code, and transfer the firmware to the development board.

2.1 STM32 MICROCONTROLLER OVERVIEW

STM32 is a family of 32-bit microcontrollers developed by STMicroelectronics. These microcontrollers are based on ARM processor architectures and are widely used in embedded systems, industrial automation, consumer electronics, robotics, and security applications. STM32 microcontrollers provide processing capabilities, memory, timers, communication interfaces, and general-purpose input/output pins for interfacing with external electronic components. The STM32F103C8T6 belongs to the STM32F1 series and is based on the ARM Cortex-M3 processor core. It is suitable for embedded applications that require reliable processing, digital input and output operations, and communication with external sensors and devices. In the proposed Smart Door Security System, the STM32F103C8T6 acts as the central processing unit. It receives input signals from the keypad and PIR sensor, processes the entered password, and controls the connected output devices according to the programmed logic.

The microcontroller performs the following functions:

1. Reads the keys pressed on the 4×4 keypad.
2. Compares the entered password with the predefined password.
3. Controls LED indicators to display the system status.
4. Activates the buzzer when a programmed alert condition occurs.
5. Generates control signals for an optional servo motor if automatic door operation is implemented.

2.2 STM32 DEVELOPMENT BOARD AND SPECIFICATIONS

The STM32F103C8T6 is commonly used on a compact development board known as the Blue Pill. The board provides access to the microcontroller's pins and includes supporting circuitry that simplifies the development of embedded applications. The development board allows the microcontroller to be connected to sensors, input devices, and output components without requiring a custom printed circuit board. It is therefore suitable for academic projects and experimental prototyping.

Technical Specifications

Parameter	Specification
Microcontroller	STM32F103C8T6
Manufacturer	STMicroelectronics
Processor core	ARM Cortex-M3, 32-bit
Maximum CPU frequency	72 MHz
Flash memory	64 KB nominal
SRAM	20 KB
Operating voltage	Approximately 2.0 – 3.6 V
GPIO capability	Up to 37 general-purpose I/O pins, depending on configuration
Analog-to-digital converter	12-bit ADC
Communication interfaces	USART, SPI, I²C, USB, and CAN peripherals
Timers	Multiple general-purpose and advanced-control timers
Development environment	STM32CubeIDE
Programming language	Embedded C

Figure 2.1: Technical specifications table

The STM32 development board provides the interface between the microcontroller and the external components used in the project. The keypad and PIR sensor supply input signals, while the LEDs and buzzer provide visual and audible feedback. The board's GPIO pins can be configured as inputs or outputs according to the requirements of each connected component. Timers and other peripherals may be configured when additional timing or control functions are required. The microcontroller operates primarily at 3.3 V logic levels.

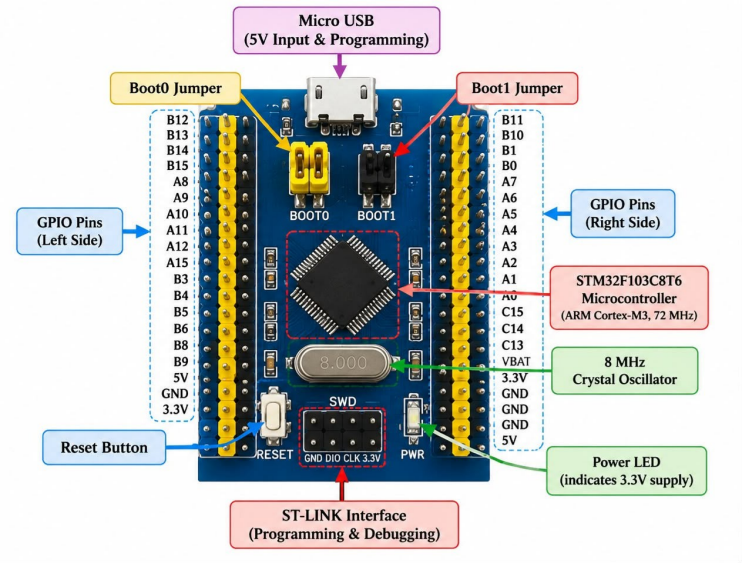


Figure 2.2: STM32F103C8T6 Development Board (Blue Pill)

2.3 HARDWARE COMPONENTS AND SENSORS USED

The Smart Door Security System uses several electronic components to implement password entry, motion detection, status indication, and alert generation.

2.3.1 STM32F103C8T6 Development Board

The STM32F103C8T6 development board is the main processing unit of the system.It executes the embedded program and coordinates communication between the input and output components.It receives keypad inputs, processes the password, monitors the PIR sensor signal, and controls the LEDs and buzzer. If a servo motor is included, the microcontroller can also generate the appropriate control signal.

2.3.2 4×4 Keypad

A 4×4 keypad consists of 16 keys arranged in four rows and four columns. It provides a convenient interface for entering numeric or alphanumeric information. In this project, the keypad is used to enter the predefined security password.

2.3.3 LED Indicators

Light-emitting diodes (LEDs) are used to provide visual feedback about the operating state of the system. Different colours can be assigned to indicate successful password verification, an unsuccessful access attempt, or another defined condition.For example, a green LED may indicate successful authentication, while a red LED may indicate an incorrect password or warning condition.Each LED must be connected with an appropriate current-limiting resistor to prevent excessive current. The GPIO current limits of the microcontroller must also be respected.

2.3.4 Power Supply

The power supply provides the electrical energy required by the microcontroller and the connected components. A stable and suitable supply is essential for reliable system operation.The STM32F103C8T6 operates at low voltage, and its GPIO pins use 3.3 V logic levels. The development board may include a voltage regulator, but the permitted input voltage depends on the specific board design.

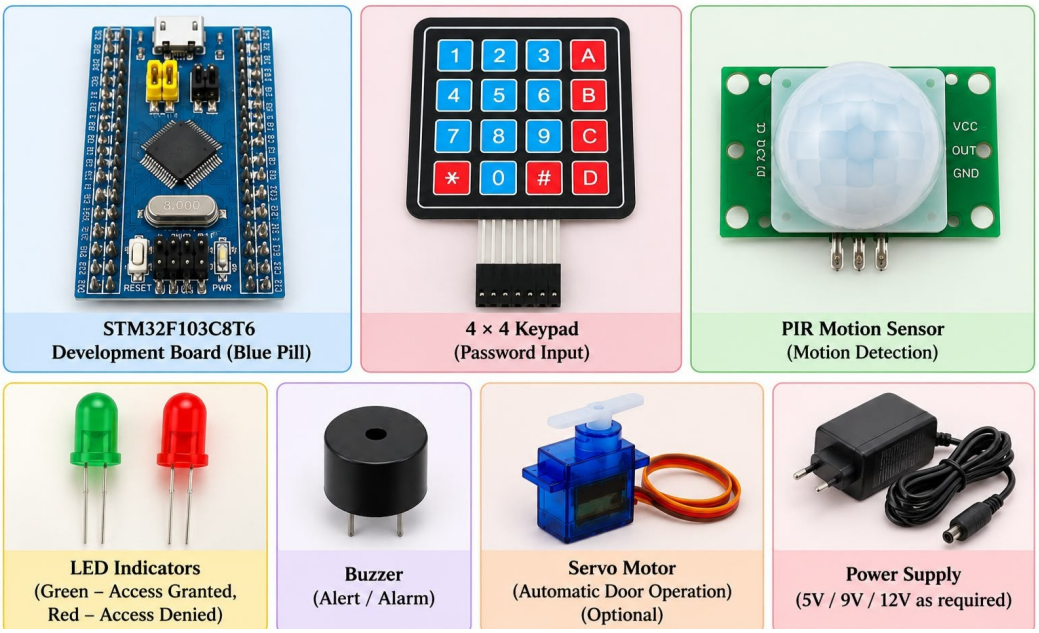


Figure 2.3: Hardware Components Used in the Smart Door Security System

The keypad, PIR sensor, LEDs, and buzzer must be supplied according to their individual electrical specifications. If a servo motor is included, it may require a separate suitable supply because its current demand can exceed the capacity of the development board's regulator.

Component	Purpose
STM32F103C8T6	Processes inputs and controls outputs.
4×4 keypad	Accepts password input.
PIR sensor	Detects movement near the entrance.
Green LED	Indicates successful authentication.
Red LED	Indicates a warning or unsuccessful attempt.
Buzzer	Produces audible alerts.
Power supply	Provides suitable operating power.
Servo motor (optional)	Controls automatic door movement.

Figure 2.4: Hardware Summary Table

2.4 SOFTWARE REQUIREMENTS

Software tools are required to develop, compile, configure, program, and test the Smart Door Security System. The selected development environment supports embedded C programming and provides facilities for configuring the STM32 peripherals.

2.4.1 Operating System

A compatible operating system is required to install and run the development software. Windows 11 is used as the development environment for this project.

2.4.2 STM32CubeIDE

STM32CubeIDE is an integrated development environment provided by STMicroelectronics for STM32 microcontroller development.

2.4.3 Embedded C Programming Language

Embedded C is used to implement the control logic of the security system. It allows the programmer to define the behaviour of the microcontroller and interface with its peripherals. The program includes functions for initialising the microcontroller, reading keypad inputs, checking the password, monitoring the PIR sensor, and controlling the output devices.

2.4.4. STM32 Firmware and Configuration Tools

STM32 firmware libraries and configuration tools support peripheral initialisation and hardware configuration. Depending on the project setup, STM32CubeMX functionality can be used to configure GPIO pins, clocks, timers, and other peripherals and generate initialisation code.

2.4.5 Device Programming and Debugging Tools

A compatible programming interface is required to transfer the compiled firmware to the microcontroller and, where supported, debug its execution. An ST-LINK programmer/ debugger is a common option for programming STM32 development boards through the Serial Wire Debug (SWD) interface.

Software or Tool	Purpose
Windows 11	Provides the operating system for software development.
STM32CubeIDE	Used for writing, compiling, programming, and debugging the embedded application.
Embedded C	Implements the control logic for password verification, motion detection, and alert generation.
STM32 HAL Library	Provides predefined functions for controlling microcontroller peripherals.
STM32CubeMX Functionality	Configures GPIO pins, clocks, timers, and other peripherals and generates initialization code.
ST-LINK Programmer/Debugger	Transfers firmware to the STM32 microcontroller and supports debugging through the SWD interface.

Figure 2.5: Software Summary Table

2.5 STM32CubeIDE AND STM32 HAL LIBRARY

2.5.1 STM32CubeIDE

STM32CubeIDE is an integrated development environment designed for STM32 microcontroller applications. It combines source-code editing, building, programming, and debugging tools within a single environment. The software provides a structured workflow for creating embedded applications. Source files can be organised into project folders, compiled into firmware, and programmed onto a compatible development board.

The general development process consists of the following steps:

1. Create or configure a project for the STM32F103C8T6 microcontroller.
2. Configure the required GPIO pins and peripherals.
3. Write the embedded C code for the keypad, PIR sensor, and output devices.
4. Compile the source code and resolve any errors.
5. Transfer the generated firmware to the development board using a compatible programming method.
6. Test the program and debug any issues observed during operation.

2.5.2 STM32 HAL Library

The STM32 Hardware Abstraction Layer (HAL) is a software library provided by STMicroelectronics.In this project, HAL functions may be used to configure GPIO pins, read the PIR sensor's digital output, control LEDs, and activate the buzzer. Additional timer functions may be used if PWM control is required for an optional servo motor.

Examples of relevant HAL functions include:

- "HAL_GPIO_ReadPin()" — reads the state of a configured GPIO input.
- "HAL_GPIO_WritePin()" — sets or resets a configured GPIO output.
- "HAL_GPIO_TogglePin()" — changes the state of a configured GPIO output.
- "HAL_Delay()" — provides a blocking delay in milliseconds.
- "HAL_Init()" — initialises the HAL library and the associated system-level facilities.

Advantages of Using STM32 HAL

The STM32 HAL library offers several advantages:

1. **Simplified Peripheral Control:**The HAL library provides ready-made functions for controlling peripherals such as GPIO pins, timers, UART, and other hardware modules. This makes it easier to interface input and output devices without writing all the low-level control instructions manually.

2. **Improved Code Readability:**The library uses structured functions that make programs easier to understand, modify, and maintain. This helps developers organise the program into separate sections for different operations, such as keypad input, password verification, motion detection, and alert generation.

3. **Faster Development:**The predefined functions reduce the amount of code that developers need to write from scratch. This saves development time and allows greater focus on implementing the main functionality of the Smart Door Security System.

4. **Structured Peripheral Initialisation:** The HAL library supports systematic configuration and initialisation of microcontroller peripherals. It works with STM32CubeMX functionality to configure GPIO pins, clocks, and other peripherals, helping establish the required hardware settings before program execution.

5. **Improved Portability:**HAL-based code can be adapted more easily between supported STM32 microcontrollers when the required peripherals and functions are available. This can reduce the effort needed to modify the project for compatible hardware platforms.

6. **Debugging and Error Handling Support:**The HAL library works with STM32CubeIDE to support program development, testing, and debugging. Its functions often provide status return values that help developers identify whether hardware operations have completed successfully or encountered an error.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The Smart Door Security and Access Control System is designed using the STM32F103C8T6 microcontroller as its central processing unit. The system consists of a 4×4 keypad, two LEDs, and a servo motor. The keypad allows the user to enter a predefined password, while the LEDs indicate the current security status. The servo motor controls the opening and closing of the miniature door. The microcontroller processes the keypad input, verifies the entered password, and controls the output devices according to the program logic.

The main functional units are:

- Input Unit: A 4×4 keypad used to enter the password.
- Processing Unit: STM32F103C8T6 microcontroller used to process the input and verify the password.
- Output Unit: Red and green LEDs used for status indication.
- Actuation Unit: A servo motor controlled using a PWM signal to operate the door mechanism.
- Power Supply: Provides the required power to the microcontroller and connected components.

3.2 BLOCK DIAGRAM

The block diagram represents the interaction between the input, processing, and output units of the

system. Block diagram flow:

"4×4 Keypad → STM32F103C8T6 Microcontroller → Password

Verification" The microcontroller then controls the following outputs:

- Red LED – indicates the locked state.
- Green LED – indicates successful authentication.
- Servo Motor – operates the door mechanism.

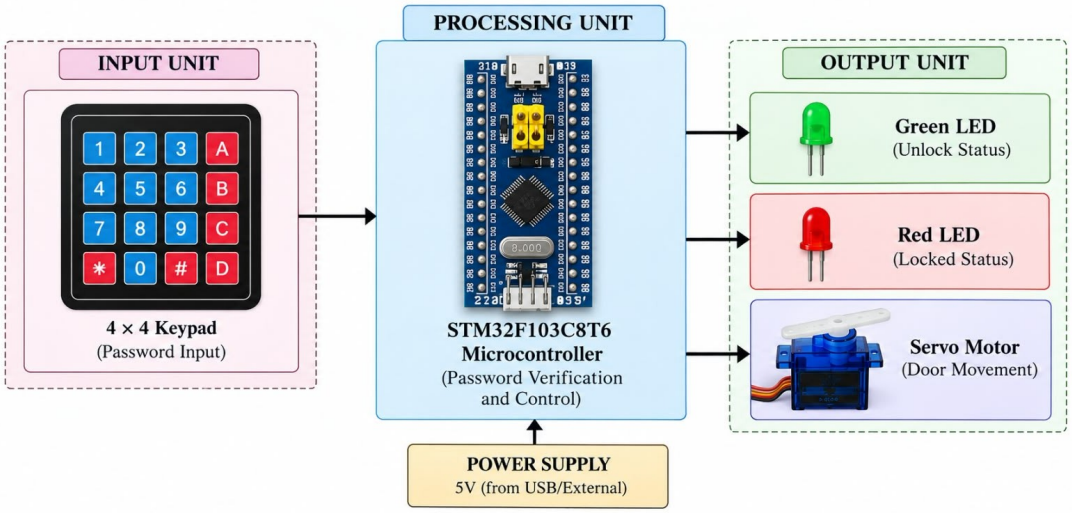


Figure 3.1: Block Diagram of the Smart Door Security and Access Control System

3.3 CIRCUIT DIAGRAM

The circuit consists of the STM32F103C8T6 microcontroller connected to a 4×4 keypad, red and green LEDs, and a servo motor. The keypad is connected to GPIO pins PB0, PB1, PB10, PB11, PB12, PB13, PB14, and PB15. The red LED is connected to PA8, while the green LED is connected to PA0. Both LEDs use current-limiting resistors. The servo motor's control signal is connected to PA6, configured as TIM3_CH1 for PWM generation. The components share a common ground reference. The servo motor should receive a suitable power supply according to its specifications.

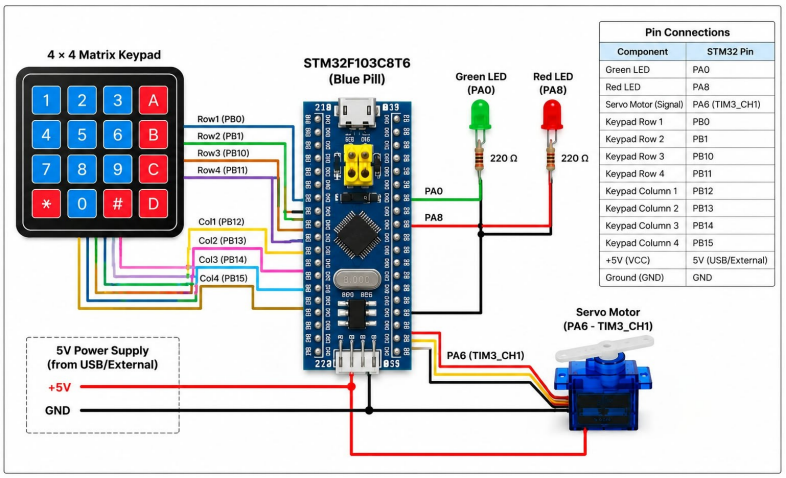


Figure 3.2: Circuit Diagram of the STM32-Based Smart Door Lock System

3.4 HARDWARE COMPONENT DESCRIPTION

4..1 STM32F103C8T6 Microcontroller

The STM32F103C8T6 is an ARM Cortex-M3-based microcontroller that acts as the main controller of the system. It reads keypad inputs, executes the password verification logic, controls the LEDs, and generates the PWM signal required to operate the servo motor.

4.2 4×4 Matrix Keypad

The 4×4 matrix keypad consists of sixteen keys arranged in four rows and four columns. It provides the interface for entering the security password. The "#" key is used to submit the entered password, while the "*" key clears the current input

4.3 Red LED

The red LED indicates the locked state of the system. It remains ON when the door is locked and turns OFF when the correct password is accepted.

4.4 Green LED

The green LED indicates successful password authentication and the unlocking process. It turns ON when the correct password is entered.

4.5 Servo Motor

The servo motor operates the miniature door mechanism. It receives a PWM control signal from the STM32 microcontroller through PA6 (TIM3_CH1). The programmed control values determine the servo position for the unlocking and locking operations.

4.6 Power Supply

The microcontroller and peripheral components require a suitable power source. The servo motor must receive a supply compatible with its voltage and current requirements. A common ground must be maintained between the servo supply and the microcontroller.

3.5 PIN CONFIGURATION AND PERIPHERAL MAPPING

The GPIO pins and timer peripheral are configured according to the requirements of the system.

Component	STM32 Pin	Configuration	Function
Green LED	PA0	GPIO Output	Unlock status indication
Red LED	PA8	GPIO Output	Locked status indication
Servo Motor	PA6	TIM3_CH1 PWM	Door movement control
Keypad Row 1	PB0	GPIO Input/Output as implemented	Keypad scanning
Keypad Row 2	PB1	GPIO Input/Output as implemented	Keypad scanning
Keypad Row 3	PB10	GPIO Input/Output as implemented	Keypad scanning
Keypad Row 4	PB11	GPIO Input/Output as implemented	Keypad scanning
Keypad Column 1	PB12	GPIO Input/Output as implemented	Keypad scanning
Keypad Column 2	PB13	GPIO Input/Output as implemented	Keypad scanning
Keypad Column 3	PB14	GPIO Input/Output as implemented	Keypad scanning
Keypad Column 4	PB15	GPIO Input/Output as implemented	Keypad scanning

Figure 3.3: Pin Configuration and Peripheral Mapping Table

3.6 ALGORITHM AND FLOWCHART

3.6.1 Algorithm

1. Start the system.
2. Initialize the GPIO pins and timer peripheral.
3. Turn ON the red LED and turn OFF the green LED.
4. Wait for the user to enter the password using the keypad.
5. If the "*" key is pressed, clear the entered password.
6. If the "#" key is pressed, compare the entered password with the stored password.
7. If the password is correct:
   7.1 Turn OFF the red LED.
   7.2 Turn ON the green LED.
   7.3 Operate the servo motor to unlock the door.
   7.4 Wait for the programmed delay.
   7.5 Return the servo motor to the locked position.
   7.6 Restore the locked-state LED indications.

8. If the password is incorrect, keep the door locked and the red LED ON.

9. Return to password entry.

10. Repeat the process.

3.6.2 Flowchart

The flowchart should represent the following sequence:

Start → Initialise GPIO and PWM → Set Locked State → Enter Password → Press "#"? → Verify Password

→ Correct?

- Yes: Green LED ON, red LED OFF → Servo unlocks door → Delay → Servo returns to locked position → Restore locked-state LEDs.
- No: Keep door locked → Red LED ON → Return to password entry.

The "*" key clears the entered password and allows the user to start entering it again.

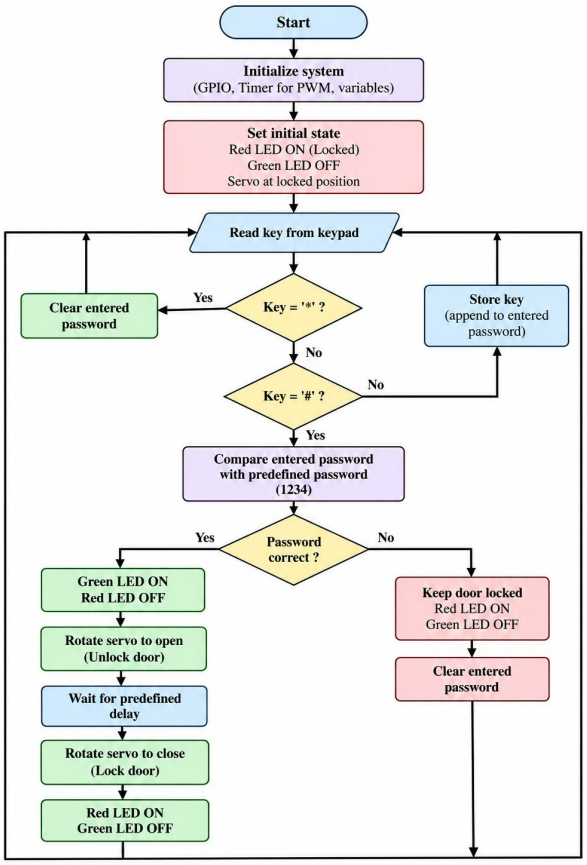


Figure 3.4: Flowchart of the Password-Based Door Lock System

3.7 WORKING PRINCIPLE

When the system is powered ON, the STM32 microcontroller initialises the required GPIO pins and PWM timer. The red LED turns ON to indicate that the door is locked, while the green LED remains OFF.The user enters the password using the 4×4 keypad. The microcontroller reads the entered characters and checks the password when the "#" key is pressed. The "*" key can be used to clear the entered characters.If the entered password matches the predefined password, "1234", the microcontroller turns OFF the red LED and turns ON the green LED. It then generates a PWM signal through PA6 to move the servo motor and unlock the miniature door.After the programmed delay, the servo motor returns to its locked position, and the LED indications return to the locked state.If an incorrect password is entered, the system does not activate the unlocking mechanism, and the door remains locked.Thus, the system demonstrates password-based access control using a microcontroller, keypad, LED indicators, and servo motor.

CHAPTER 4

IMPLEMENTATION

4.1 Development Environment Setup

The Smart Door Lock System was developed using STM32CubeIDE, an integrated development environment for programming STM32 microcontrollers. The STM32F103C8T6 microcontroller was selected as the main controller. The project was created in STM32CubeIDE, and the required embedded C source code was developed and compiled. The program was built successfully without compilation errors.

4.2 STM32CubeMX Configuration

STM32CubeMX configuration tools integrated into STM32CubeIDE were used to configure the microcontroller pins and peripherals. The GPIO pins were assigned for keypad interfacing and LED control. Timer 3 was configured to generate the PWM signal required to control the servo motor connected to pin PA6.The system clock was configured using the internal high-speed oscillator (HSI). The generated initialisation code was used as the foundation for developing the application.

4.3 GPIO AND PERIPHERAL CONFIGURATION

The GPIO pins and timer peripheral were configured according to the hardware requirements.

Component	STM32 Pin	Function
 Green LED	PA0	Indicates successful password verification
 Red LED	PA8	Indicates the locked state
 Keypad Rows	PB0, PB1, PB10, PB11	Keypad row scanning
 Keypad Columns	PB12, PB13, PB14, PB15	Keypad input detection
 Servo Motor	PA6	PWM-based door movement

Figure 4.1: STM32 Pin Configuration and Component Functions

Timer 3 was configured in PWM mode to generate the control signal for the servo motor. The keypad GPIO pins were configured to support row scanning and column detection.

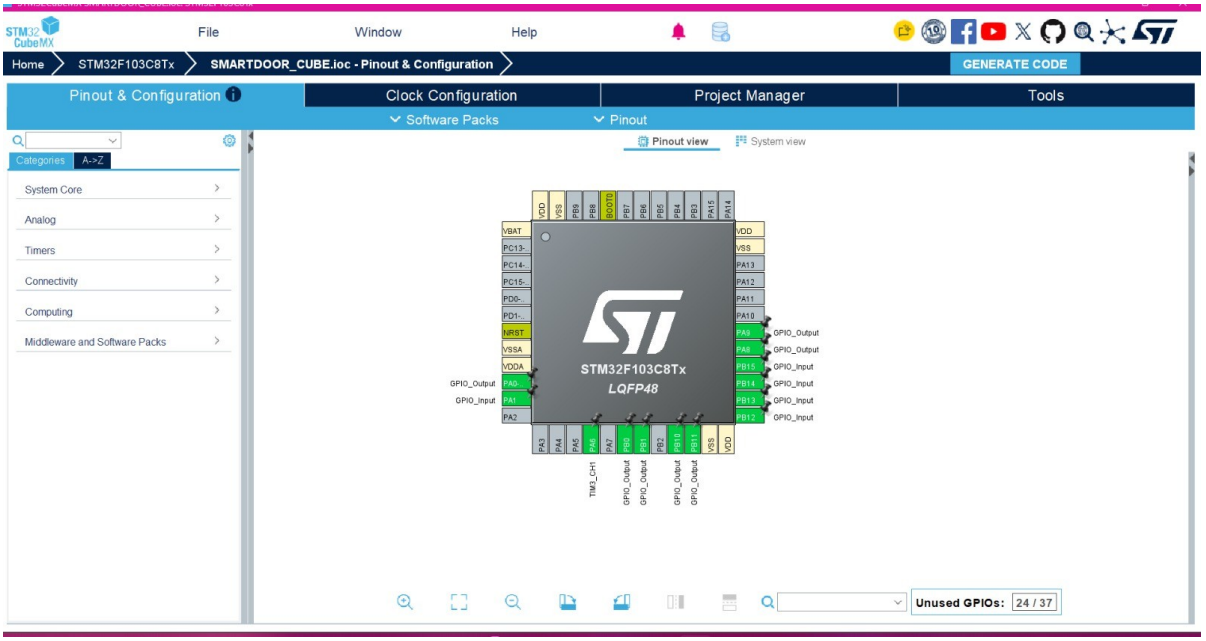


Figure 4.2: Pinout configuration of STM32F103C8T6

4.4 EMBEDDED C PROGRAM DEVELOPMENT

The application was developed in Embedded C. The program initialises the GPIO pins and timer peripheral before continuously scanning the keypad for user input.The password is stored in the program as "1234". When the user enters the password and presses the "#" key, the entered sequence is compared with the stored password. The "*" key is used to clear the entered sequence.If the password is correct, the green LED turns ON, the red LED turns OFF, and the servo motor moves to the door-opening position. After a predefined delay, the servo returns to the closing position, and the LEDs return to their locked-state indication.If the password is incorrect, the door remains locked and the red LED indicates the locked state.

4.5 HARDWARE INTERFACING AND INTEGRATION

The STM32F103C8T6 microcontroller was interfaced with a 4×4 membrane keypad, a red LED, a green LED, and a servo motor. The LEDs were connected through current-limiting resistors. The keypad was connected to the designated GPIO pins, while the servo signal wire was connected to PA6.

The hardware components were assembled on a prototype base, and the servo motor was mechanically connected to the miniature door. The components were integrated to demonstrate password-based access control and automatic door movement.

4.6 PROGRAM EXECUTION AND DEBUGGING

The embedded C program was compiled using STM32CubeIDE. After successful compilation, the generated firmware was programmed into the microcontroller using the available programming interface.The prototype was tested by entering the configured password through the keypad. The LED indications and servo movement were observed to verify the operation of the access-control system. Hardware connections and component polarity were checked during testing to resolve initial issues with LED indication and servo operation.

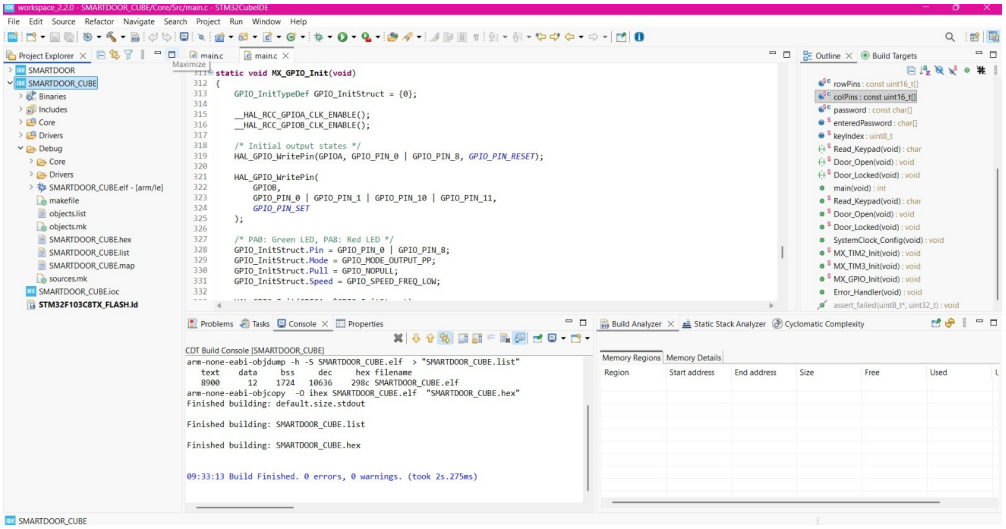


Figure 4.3:Firmware Programming Using STM32CubeProgrammer

4.7 IMPLEMENTATION CHALLENGES AND SOLUTIONS

During implementation, a few hardware-related issues were encountered.

- LED polarity: Incorrect LED orientation prevented the expected indication. The LED connections were corrected by connecting the appropriate terminals through current-limiting resistors.

- Keypad interfacing: The keypad wiring was checked and corrected to match the GPIO pin assignments in the program.
- Servo operation: The servo signal connection and mechanical linkage were adjusted to improve door movement.
- Hardware integration: The connections between the microcontroller and peripheral components were checked to ensure reliable operation.

These corrections helped achieve the intended operation of the password-controlled door-lock prototype.

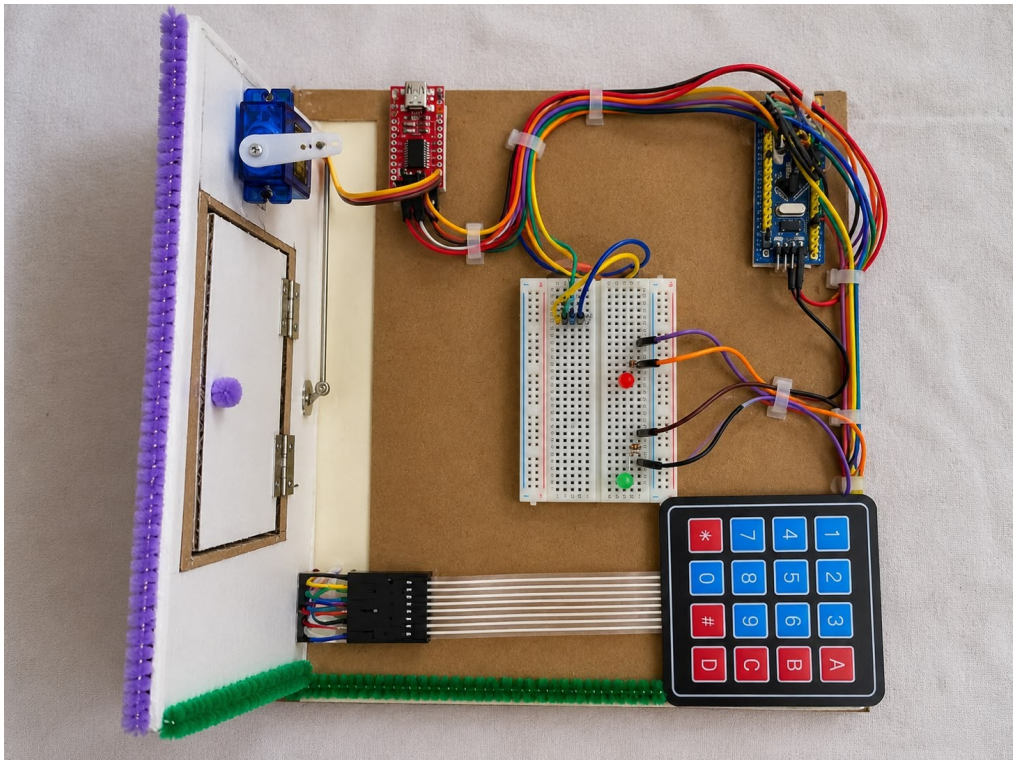


Figure 4.4: password-controlled door-lock prototype.

CHAPTER 5

TESTING AND RESULTS

5.1 TESTING METHODOLOGY

The Smart Door Lock System was tested to verify password authentication, LED indications, keypad input, and servo-controlled door movement. The system was powered on, and the keypad was used to enter the predefined password. The responses of the LEDs and servo motor were observed to evaluate the operation of the prototype.

5.2 TEST CASES AND OBSERVATIONS

The following tests were considered for evaluating the system.

Test Case	Input or Action	Expected Result
System startup	Power ON	Red LED indicates the locked state
Correct password	Enter "1234#"	Green LED turns ON, red LED turns OFF, and the servo operates
Incorrect password	Enter an incorrect password followed by "#"	Door remains locked
Clear input	Press "*"	Entered password is cleared
Door movement	Enter the correct password	Servo moves to the opening position and returns to the locked position

Figure 5.1: System Test Cases and Expected Results

5.3 HARDWARE IMPLEMENTATION PHOTOGRAPHS

Photographs of the completed prototype are included to demonstrate the physical implementation of the Smart Door Lock System. The images show the STM32 microcontroller, keypad, red and green LEDs, servo motor, and miniature door mechanism.

5.4 OUTPUT SCREENSHOTS

The STM32CubeIDE build output can be included as evidence of successful compilation. A screenshot showing zero compilation errors demonstrates that the embedded C program was built successfully.

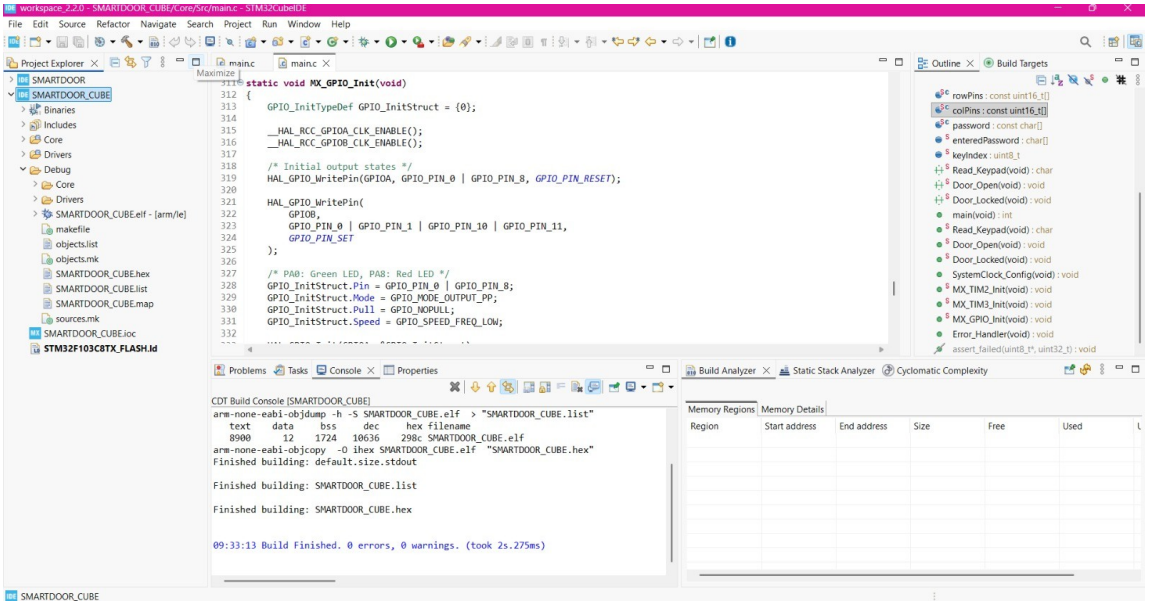


Figure 5.2:STM32CubeIDE Build Output

5.5 RESULT ANALYSIS

The prototype demonstrates password-based access control using the STM32F103C8T6 microcontroller. The keypad provides password input, the embedded program verifies the entered password, the LEDs indicate the system state, and the servo motor controls the door movement.

The implementation demonstrates the integration of GPIO programming, keypad interfacing, PWM-based servo control, and embedded C programming in a practical security application. The prototype provides a foundation for further improvements in electronic door-locking systems.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

The project, “Smart Door Security and Access Control System Using STM32,” was successfully developed as a password-based door access control prototype using the STM32F103C8T6 microcontroller. The system uses a 4×4 keypad for password entry, LEDs to indicate the security status, and a servo motor to control the door mechanism.When the correct password is entered, the system unlocks the door by operating the servo motor and changes the LED indication. After a programmed delay, the door returns to the locked position. Incorrect password entries do not activate the unlocking mechanism.The project demonstrates the practical application of embedded C programming, GPIO configuration, PWM generation, and hardware interfacing. It provides a simple and cost-effective prototype for understanding microcontroller-based access control systems.

6.2 LEARNING OUTCOMES

- Understanding the STM32F103C8T6 microcontroller and its features.
- Creating and configuring projects using STM32CubeIDE and STM32CubeMX.
- Programming GPIO pins for input and output operations.
- Interfacing a 4×4 keypad with a microcontroller.
- Controlling LEDs for status indication.
- Generating PWM signals to control a servo motor.
- Writing and debugging embedded C programs.
- Identifying and correcting hardware connection and integration issues.
- Understanding the importance of testing both hardware and software.

6.3 LIMITATIONS

- Access is controlled using a fixed password stored in the program.
- The system does not include a display to show access status or password entry.

- The prototype does not have Wi-Fi or Bluetooth connectivity for remote monitoring.
- There is no dedicated alarm for repeated incorrect password attempts.
- The servo-operated door mechanism is suitable for a miniature prototype rather than a full-sized door.
- The system has not been designed with advanced security features such as encrypted authentication or tamper detection.

6.4 FUTURE ENHANCEMENTS

The project can be improved by incorporating additional features in future versions:

- LCD or OLED Display: To show password entry, access status, and error messages.
- Fingerprint Authentication: To provide an additional method of user verification.
- IoT Connectivity: To enable remote monitoring and access notifications through a mobile application.
- Buzzer Alarm: To alert users after repeated incorrect password attempts.
- Password Management: To allow authorised users to change the password.
- Multiple User Access: To support different passwords for multiple users.
- Tamper Detection: To detect unauthorised attempts to interfere with the system.
- Power Backup: To maintain operation during a power failure.

These enhancements could improve the security, usability, and reliability of the system. The prototype provides a foundation for developing more advanced embedded access control solutions.

APPENDIX A

COMPLETE EMBEDDED C SOURCE CODE

The complete Embedded C source code developed for the STM32-based Smart Door Security and Access Control System. The program configures the STM32F103C8T6 microcontroller to interface with a 4×4 keypad, red and green LEDs, and a servo motor. The program verifies the password entered through the keypad. When the correct password ("1234") followed by the "#" key is entered, the system activates the unlocking mechanism using PWM-based servo control and changes the LED indications. The "*" key clears the entered password. Incorrect passwords do not activate the unlocking mechanism.

Source file: "main.c"

Development environment: STM32CubeIDE

Programming language: Embedded C

Microcontroller: STM32F103C8T6

```
/* USER CODE BEGIN Header */
/**
  **************************
  * @file           : main.c
  * @brief          : STM32 Smart Door Security System
  **************************
  */
/* USER CODE END Header */
#include "main.h"
#include <string.h>
#include <stdint.h>
```

```
/* Private variables ---------------------------------------------------------*/
TIM_HandleTypeDef htim2;
TIM_HandleTypeDef htim3;
/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM2_Init(void);
static void MX_TIM3_Init(void);
void Error_Handler(void);

/* USER CODE BEGIN PV */
static const char keymap[4][4] = {
    {'1','2','3','A'},
    {'4','5','6','B'},
    {'7','8','9','C'},
    {'*','0','#','D'}
};
/* Keypad rows: PB0, PB1, PB10, PB11 */
/* Keypad columns: PB12, PB13, PB14, PB15 */
static const uint16_t rowPins[4] = {
    GPIO_PIN_0,
    GPIO_PIN_1,
    GPIO_PIN_10,
    GPIO_PIN_11
};
static const uint16_t colPins[4] = {
    GPIO_PIN_12,
    GPIO_PIN_13,
    GPIO_PIN_14,
    GPIO_PIN_15
};
```

```
static const char password[] = "1234";
static char enteredPassword[5] = {0};
static uint8_t keyIndex = 0;
/* USER CODE END PV */
/* USER CODE BEGIN PFP */

static char Read_Keypad(void);
static void Door_Open(void);
static void Door_Locked(void);

/* USER CODE END PFP */

int main(void)
{
   HAL_Init();
  SystemClock_Config();
   MX_GPIO_Init();
   MX_TIM2_Init();
   MX_TIM3_Init();
   /* Initial LED state */
   HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);
   HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
   /* Start servo PWM on PA6 (TIM3_CH1) */
   HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
   __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 1000);
 while (1)
   {
      char key = Read_Keypad();
      if (key == '\0')
      {
         continue;
      }
```

```
  if (key == '*')
      {
        keyIndex = 0;
        memset(enteredPassword, 0, sizeof(enteredPassword));
      }
      else if (key == '#')
      {
        enteredPassword[keyIndex] = '\0';
        if (strcmp(enteredPassword, password) == 0)
        {
          Door_Open();
        }
        else
        {
          Door_Locked();
        }
        keyIndex = 0;
        memset(enteredPassword, 0, sizeof(enteredPassword));
      }
      else if (key >= '0' && key <= '9' && keyIndex < 4)
      {
        enteredPassword[keyIndex++] = key;
      }
    }
}
/* USER CODE BEGIN 0 */
static char Read_Keypad(void)
{
   for (uint8_t row = 0; row < 4; row++)
   {
```

```
/* Set all keypad rows HIGH */
    HAL_GPIO_WritePin(
      GPIOB,
      rowPins[0] | rowPins[1] | rowPins[2] | rowPins[3],
      GPIO_PIN_SET
    );
    /* Set the selected row LOW */
    HAL_GPIO_WritePin(
      GPIOB,
      rowPins[row],
      GPIO_PIN_RESET
    );
    HAL_Delay(2);
    for (uint8_t col = 0; col < 4; col++)
    {
      if (HAL_GPIO_ReadPin(GPIOB, colPins[col]) == GPIO_PIN_RESET)
      {
        HAL_Delay(20);
        /* Confirm the key press */
        if (HAL_GPIO_ReadPin(GPIOB, colPins[col]) == GPIO_PIN_RESET)
        {
          /* Wait for key release */
          while (HAL_GPIO_ReadPin(GPIOB, colPins[col]) == GPIO_PIN_RESET)
          {
            HAL_Delay(10);
          }
          HAL_Delay(20);
          return keymap[row][col];
        }
      }
    }
  }
  return '\0';
}
```

```
static void Door_Open(void)
{
  /* Green LED ON, red LED OFF */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_SET);
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
  /* Open servo */
  __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 2000);

  HAL_Delay(3000);

  /* Return servo to locked position */
  __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 1000);

  /* Restore locked LED state */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
}

static void Door_Locked(void)
{
  /* Green LED OFF, red LED ON */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);

  /* Servo locked position */
  __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 1000);
}

/* USER CODE END 0 */
/* System Clock Configuration: 8 MHz HSI */
```

```
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;

  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  RCC_ClkInitStruct.ClockType =
    RCC_CLOCKTYPE_HCLK |
    RCC_CLOCKTYPE_SYSCLK |
    RCC_CLOCKTYPE_PCLK1 |
    RCC_CLOCKTYPE_PCLK2;

  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
  {
    Error_Handler();
  }
}
```

```
/* TIM2 initialization */
static void MX_TIM2_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  htim2.Instance = TIM2;
  htim2.Init.Prescaler = 0;
  htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim2.Init.Period = 65535;
  htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;

  if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;

  if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;

  if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
}
```

```
/* TIM3 PWM initialization for servo */
static void MX_TIM3_Init(void)
{
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};
  htim3.Instance = TIM3;
  htim3.Init.Prescaler = 7;
  htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim3.Init.Period = 19999;
  htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_PWM_Init(&htim3) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 1000;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;

  if (HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
 HAL_TIM_MspPostInit(&htim3);
}
```

```
/* GPIO initialization */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();
  /* Initial output states */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0 | GPIO_PIN_8, GPIO_PIN_RESET);
  HAL_GPIO_WritePin(
    GPIOB,
    GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_10 | GPIO_PIN_11,
    GPIO_PIN_SET
  );

  /* PA0: Green LED, PA8: Red LED */
  GPIO_InitStruct.Pin = GPIO_PIN_0 | GPIO_PIN_8;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
  /* PA1: PIR input (not used in this demo) */
  GPIO_InitStruct.Pin = GPIO_PIN_1;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
  /* Keypad row outputs */
  GPIO_InitStruct.Pin =
    GPIO_PIN_0 |
    GPIO_PIN_1 |
    GPIO_PIN_10 |
    GPIO_PIN_11;
```

```
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
    /* Keypad column inputs with internal pull-ups */
    GPIO_InitStruct.Pin =
        GPIO_PIN_12 |
        GPIO_PIN_13 |
        GPIO_PIN_14 |
        GPIO_PIN_15;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
}

/* Error handler */
void Error_Handler(void)
{
    __disable_irq();
    while (1)
    {
    }
}
#ifdef USE_FULL_ASSERT

void assert_failed(uint8_t *file, uint32_t line)
{
    (void)file;
    (void)line;
}
#endif
```

APPENDIX B

ADDITIONAL CIRCUIT DIAGRAMS AND CONFIGURATION DETAILS

B.1 CIRCUIT CONNECTION DETAILS

The STM32F103C8T6 microcontroller serves as the main controller of the Smart Door Security and Access Control System. The keypad is used for password entry, LEDs indicate the access status, and a servo motor operates the door mechanism.

Pin Configuration and Mapping

Component	STM32 Pin	Configuration	Function
Green LED	PA0	GPIO Output	Unlock status indication
Red LED	PA8	GPIO Output	Locked status indication
Servo Motor	PA6	TIM3_CH1 PWM	Door movement control
Keypad Row 1	PB0	GPIO Input/Output as implemented	Keypad scanning
Keypad Row 2	PB1	GPIO Input/Output as implemented	Keypad scanning
Keypad Row 3	PB10	GPIO Input/Output as implemented	Keypad scanning
Keypad Row 4	PB11	GPIO Input/Output as implemented	Keypad scanning
Keypad Column 1	PB12	GPIO Input/Output as implemented	Keypad scanning
Keypad Column 2	PB13	GPIO Input/Output as implemented	Keypad scanning
Keypad Column 3	PB14	GPIO Input/Output as implemented	Keypad scanning
Keypad Column 4	PB15	GPIO Input/Output as implemented	Keypad scanning

Figure B.1: STM32 Pin Configuration and Component Functions

Both LEDs are connected through current-limiting resistors. The servo motor uses a PWM signal from the microcontroller to control its position.

B.2 STM32CubeMX CONFIGURATION

The microcontroller peripherals were configured using STM32CubeMX within STM32CubeIDE. The GPIO pins were configured according to their respective input and output functions. PA6 was configured for the TIM3_CH1 PWM output to control the servo motor.

B.3 PASSWORD CONFIGURATION

The system uses a predefined password of "1234". The "#" key submits the entered password, while the "*" key clears the current input. Correct password entry activates the unlocking mechanism, whereas incorrect password entry keeps the door locked.

B.4 ADDITIONAL DIAGRAMS

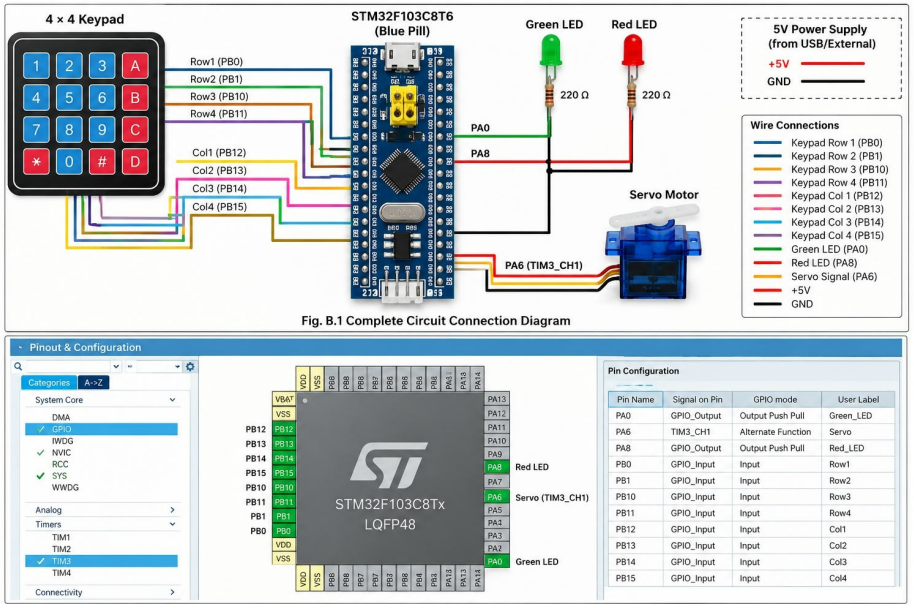


Fig. B.1 Complete Circuit Connection Diagram

Figure B.2: Complete circuit connection diagram.

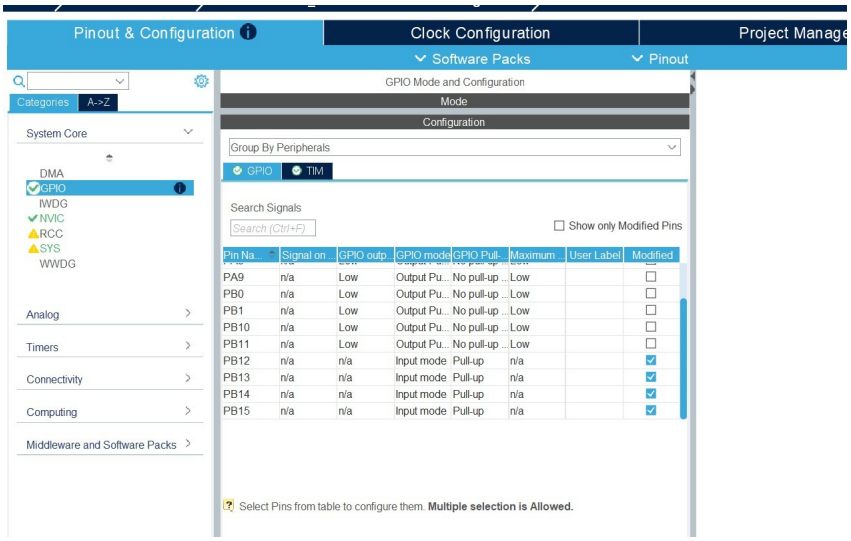


Figure B.3:STM32CubeMX pin configuration screenshot.

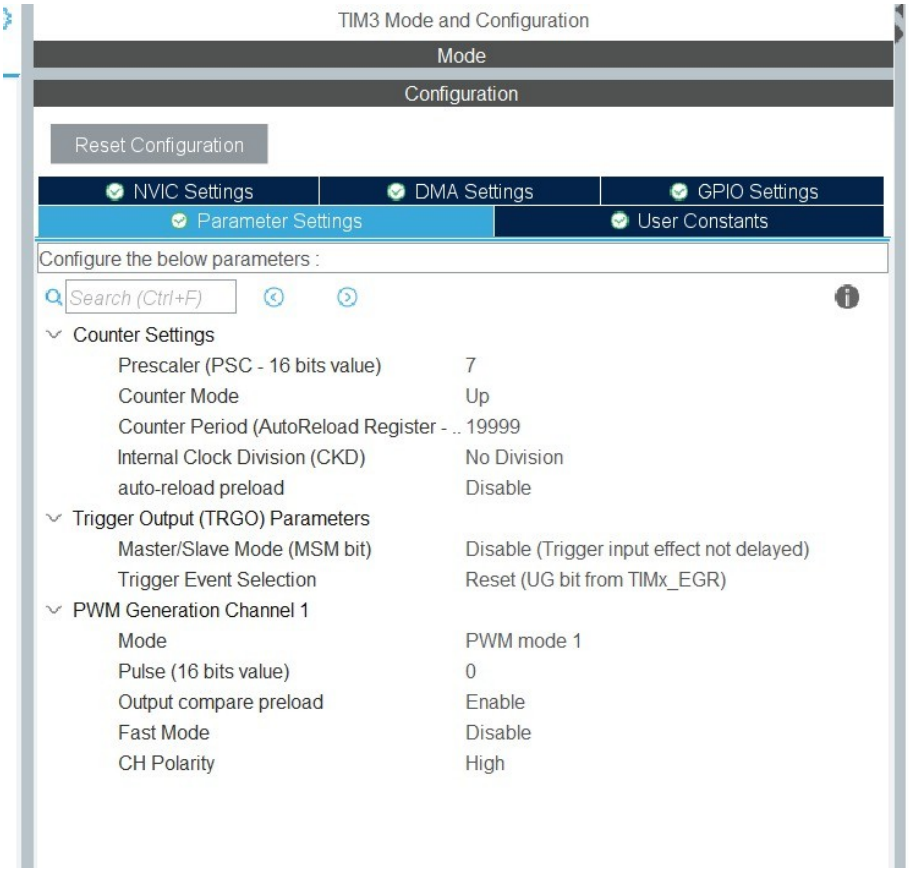


Figure B.4:TIM3 Timer Parameter Configuration

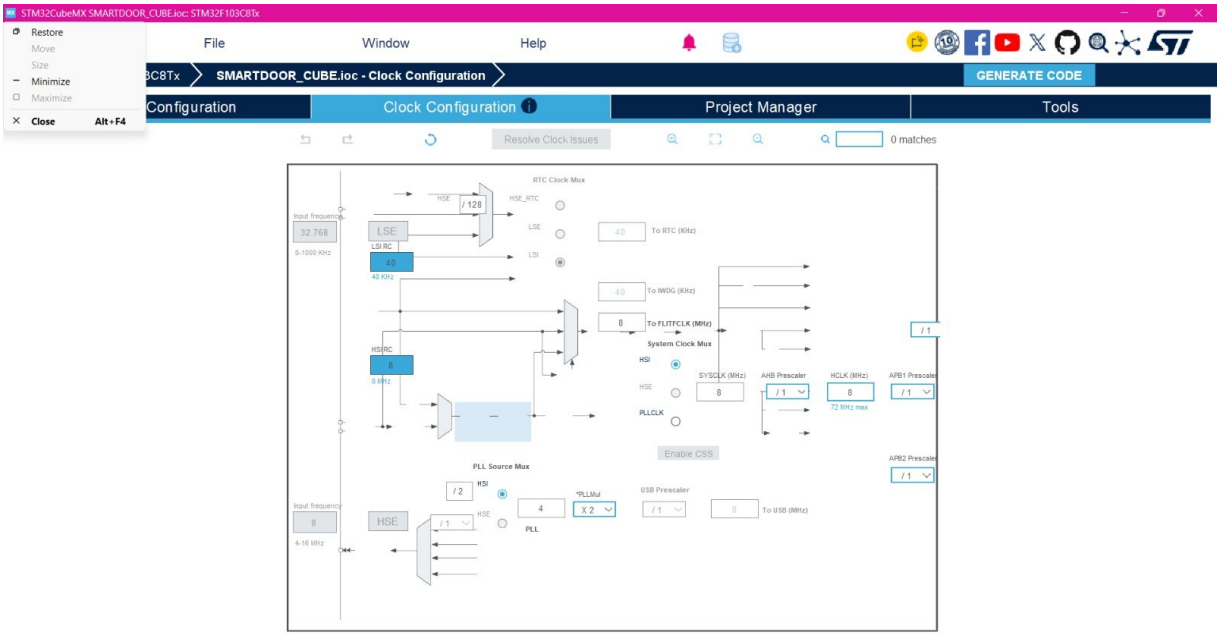


Figure B.5:Title Clock Configuration in STM32CubeMX